Bauhaus-Universität Weimar

JOHNS HOPKINS UNIVERSITY

From FEM to Neural Operators: Scientific Machine Learning for Computational Mechanics

Neural Operators

Dr-Ing Somdatta Goswami

Bauhaus Spring School 2026

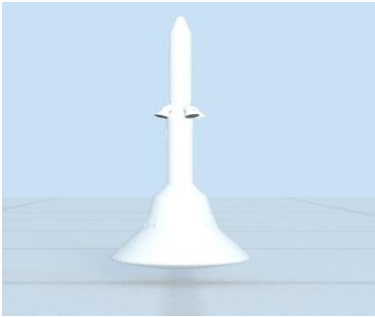
Physics-based Models

Physics-based models are approximated via **ODEs/PDEs**

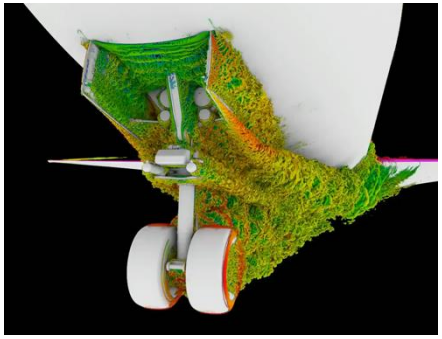
To model earthquake: $m\frac{d^2u}{dt^2} + \mathrm{k}\frac{du}{dt} + F_0 = 0$

To model waves: $\frac{\partial^2 u}{\partial t^2} - \nu^2\left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}\right) = 0$

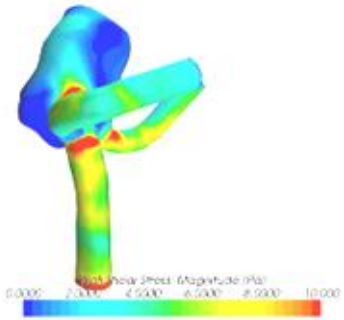
Computational Mechanics helps us simulate these equations.



Simulation of Orion Spacecraft Launch Abort System (NASA Ames)

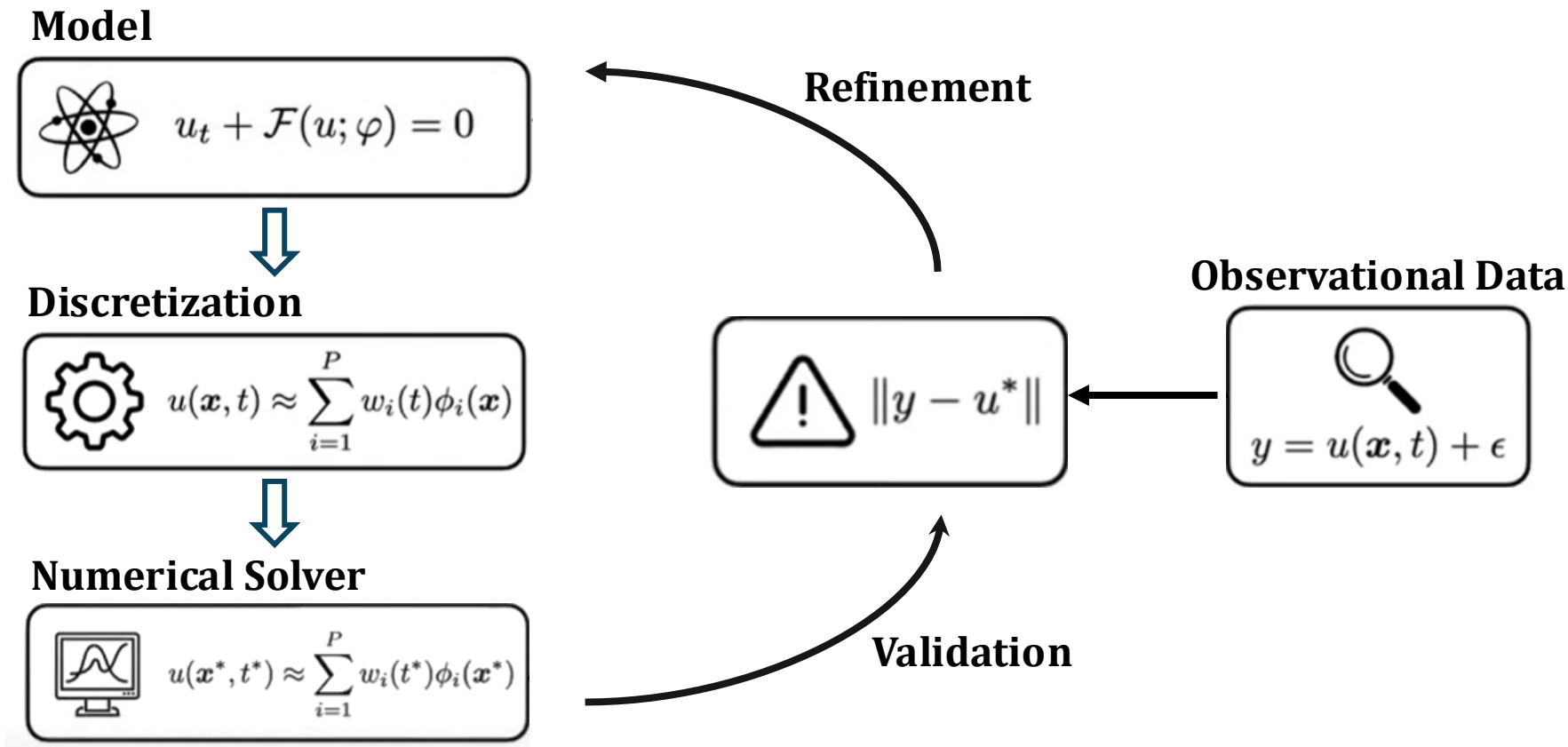


Detailed flow around an Aircraft's landing gear (NASA Ames)



CFD Simulation of a Patient-Specific Intracranial Aneurysm

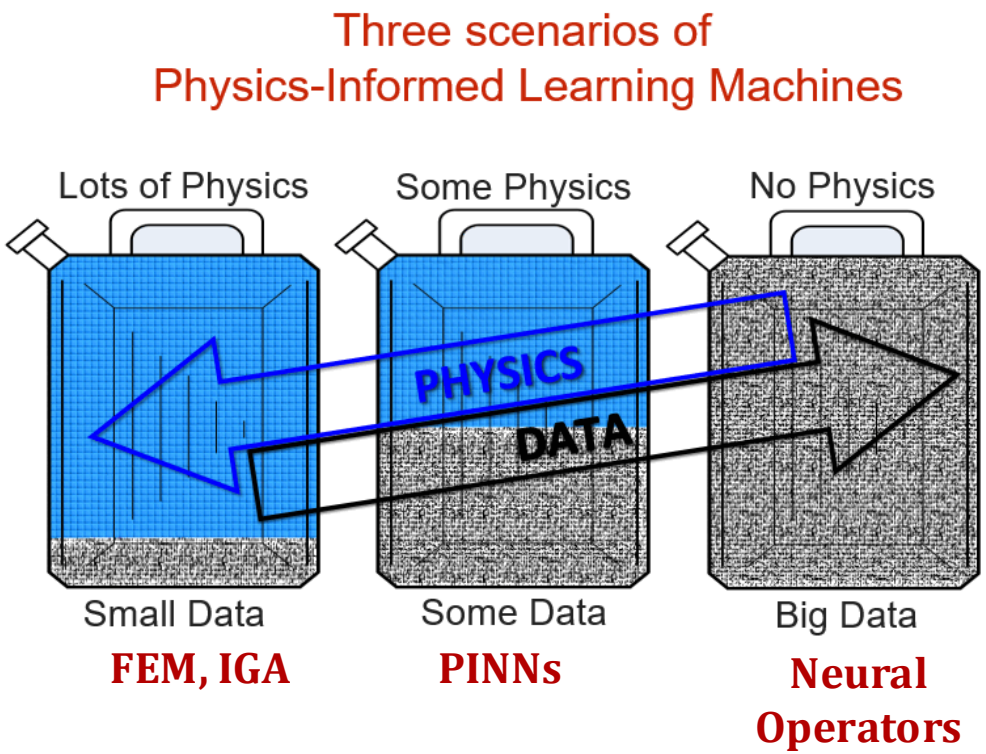
Solving a System in Computational Mechanics



Challenges

- Require knowledge of conservation laws, and boundary conditions
- Time consuming and strenuous simulations.
- Difficulties in mesh generation.
- Solving inverse problems or discovering missing physics can be prohibitively expensive.

Data + Laws of Physics

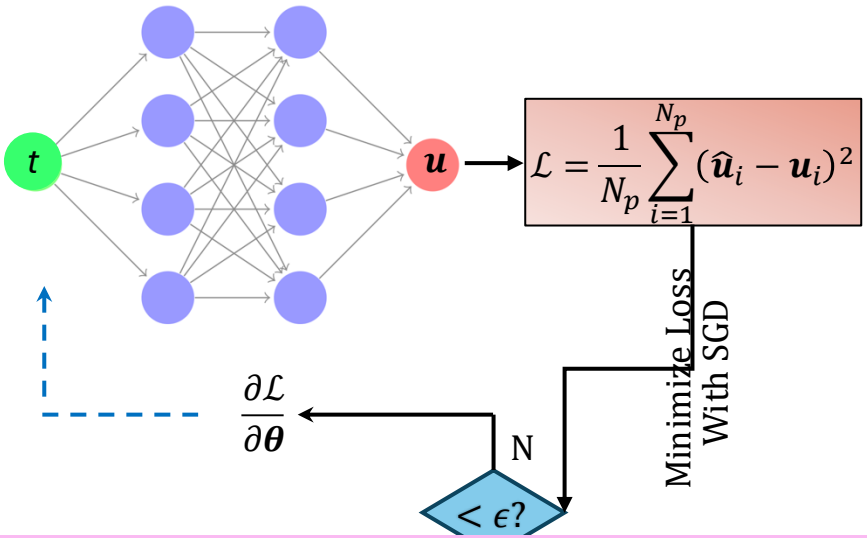


Training data-driven NNs

$$m\frac{d^2u(t)}{dt^2} + \mathrm{k}\frac{du(t)}{dt} + F = 0$$

$\mathcal{N}: t \rightarrow u(t)$ For a single F

$\hat{\boldsymbol{u}}$: Predicted solution
$\boldsymbol{\theta}$: Weights and biases
N_p: # of collocation points
$(\boldsymbol{x}, \boldsymbol{u}(\boldsymbol{x}))$: Labelled dataset



Let's say we want to learn a model for multiple F.

Training data-driven NNs

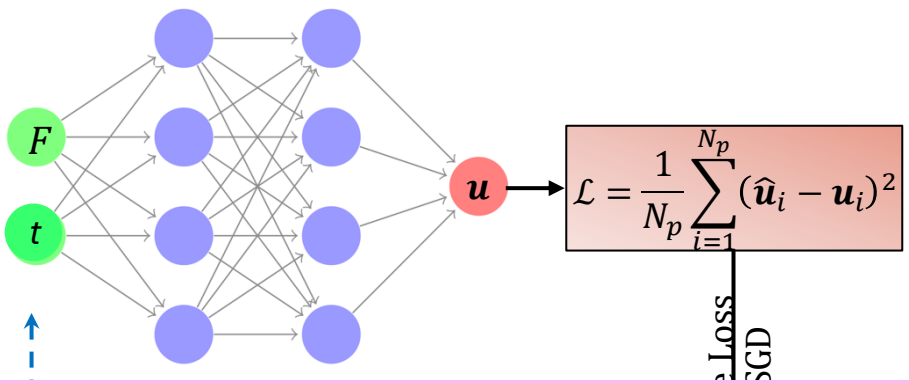
$$m\frac{d^2u}{dt^2} + \mathrm{k}\frac{du}{dt} + F = 0$$

$$\mathcal{N}: [t, F] \rightarrow u(t)$$

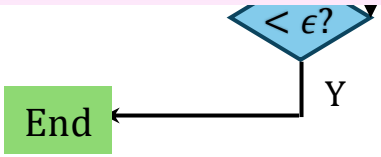
$\hat{\boldsymbol{u}}$: Predicted solution

$\boldsymbol{\theta}$: Weights and biases

N_p: # of collocation points



To learn a model for 1M Forcing functions (F), we need a very large and overparametrized model



Surrogate Modeling Techniques

- Discretized Data
- Discretization dependent
- Queries on mesh
- Learning functions between vector spaces

- Functional Data
- Discretization Invariant
- Continuous quantities
- Learning operators between function spaces

Data-driven

PCA

Auto-encoders

K-PCA

Diffusion maps

f-PCA

DeepONet

LNO

FNO

WNO

F-RKHS

Finite Dimensional

Infinite Dimensional

Physics Informed

PINNs*

PI-DeepONet

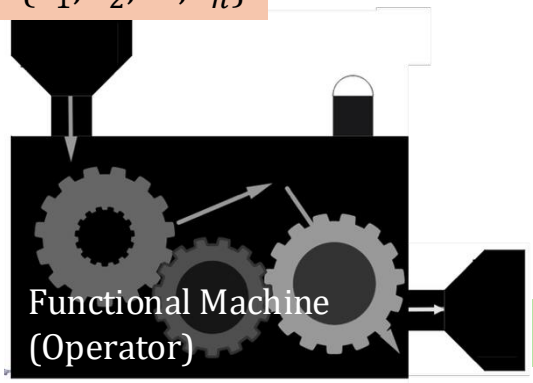
PINO

*continuous output

Learning a Neural Operator

Inputs

$\mathcal{F} = \{F_1, F_2, \dots, F_n\}$



$\Phi : \mathcal{F} \rightarrow \mathcal{U}$

$\mathcal{F}, \mathcal{S}$ are infinite dimensional function space

Outputs

$\mathcal{U} = \{u_1(\boldsymbol{F_1}), u_2(\boldsymbol{F_2}), \dots, u_n(\boldsymbol{F_n})\}$

What we know

$\{\mathcal{F}_n, \mathcal{U}_n\}_{n=1}^{N}$

$\mathcal{U}_n = \Phi(\mathcal{F}_n)\,, \mathcal{F}_n \sim \mu \; i.i.d$

Task

Operator, $\Psi : \mathcal{F} \times \Theta \rightarrow \mathcal{U}$ such that $\Psi(.\,, \theta^*) \approx \Phi$

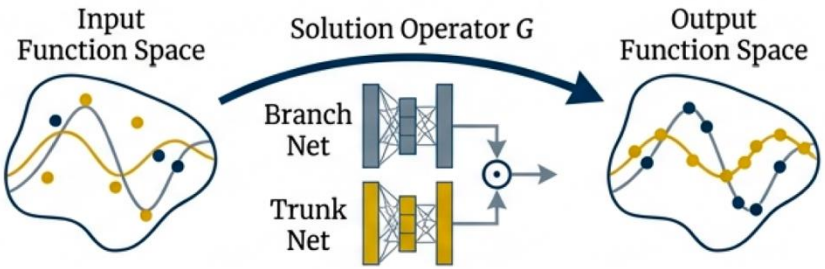
Training $\theta^* = \text{argmin}_\theta \, l(\{\mathcal{F}_n, \Psi(\mathcal{U}_n, \theta)\})$

Moving from Solving Instances to learning Operators

Traditional Solvers & Standard DL

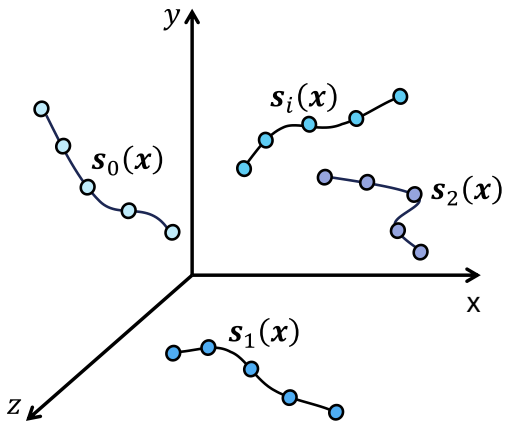
Maps a single input instance to a single output. Requires retraining for every new condition.

Neural Operators (DeepONet, FNO)



Learns the mapping between infinite-dimensional function spaces.

Operator Learning



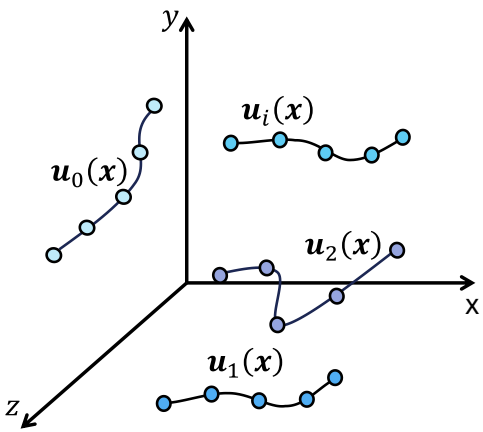
Input space

$$\Phi: \mathcal{S} \rightarrow \mathcal{U}$$

Data $\{\mathcal{S}_n, \mathcal{U}_n\}_{n=1}^{N}$
and/or Physics

Operator (Ψ)

$\Psi: \times\, \Theta \rightarrow \mathcal{U}$ such that
$\Psi(.\,, \theta^*) \approx \Phi$



Output space

Initial and Boundary conditions,
Material parameters, etc.

Displacement, Stress,
Pressure, etc.

Training

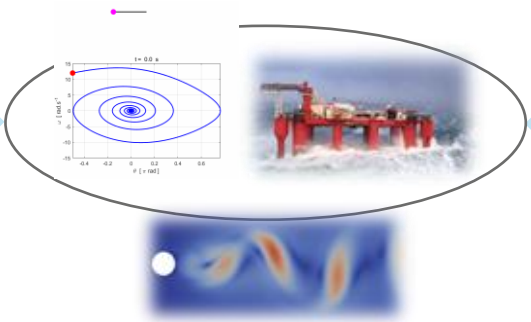
$$\theta^* = \mathrm{argmin}_{\theta}\, l(\{\mathcal{U}_n, \Psi(\mathcal{S}_n, \theta)\})$$

Deep Neural Operator

- Learning the mapping between infinite-dimensional spaces employing continuous nonlinear operators developed with DNN.
- The neural operators can learn an entire family of equations and have enhanced generalization ability.

$$G: \mathcal{F} \times \Theta \rightarrow \mathcal{S}$$

Type 1
Universal Approximation



Type 2
Green's Integral

Universal Approximation Theorem for Operators

Universal Approximation Theorem for Operator

$G : u \mapsto G(u)$, $G(u) : y \in \mathbb{R}^d \to \mathbb{R}$

Theorem (Chen & Chen, IEEE Trans. Neural Netw., 1995)

Suppose that σ is a continuous non-polynomial function, X is a Banach Space, $K_1 \subset X$, $K_2 \subset \mathbb{R}^d$ are two compact sets in X and $\mathbb{R}^d$, respectively, V is a compact set in $C(K_1)$, G is a nonlinear continuous operator, which maps V into $C(K_2)$. Then for any $\epsilon > 0$, there are positive integers n, p, m, constants $c_i^k, \xi_{ij}^k, \theta_i^k, \zeta_k \in \mathbb{R}$, $w_k \in \mathbb{R}^d$, $x_j \in K_1$, $i = 1, \ldots, n$, $k = 1, \ldots, p$, $j = 1, \ldots, m$, such that

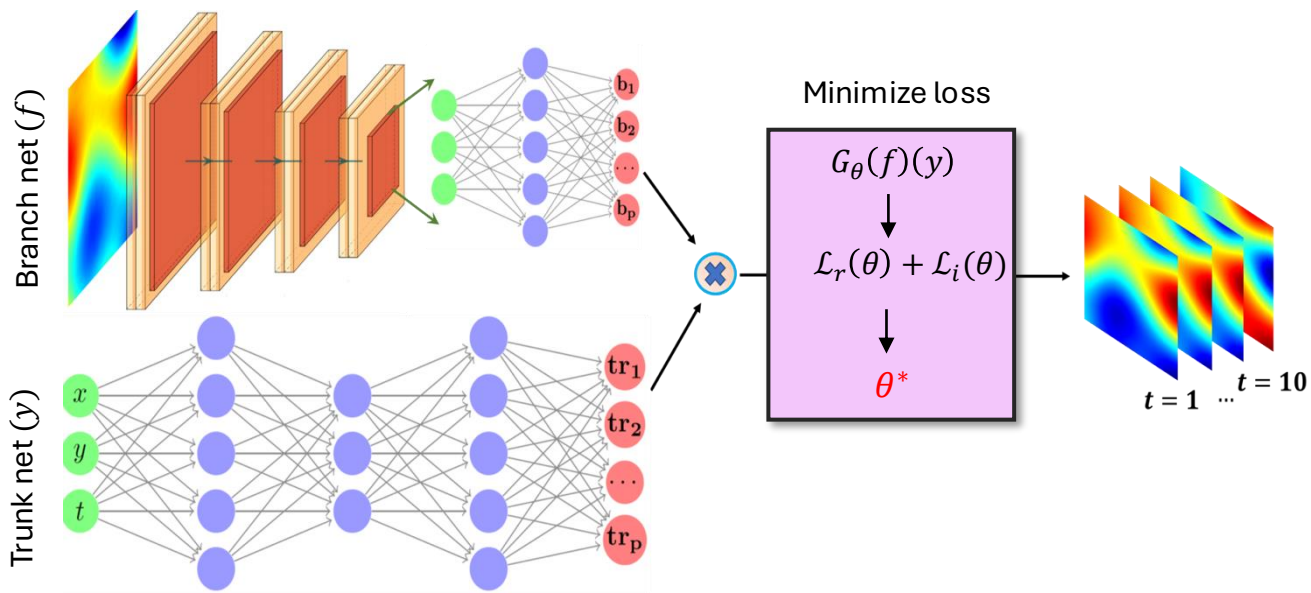
$$\left| G(u)(y) - \sum_{k=1}^{p} \underbrace{\sum_{i=1}^{n} c_i^k \sigma \left(\sum_{j=1}^{m} \xi_{ij}^k u(x_j) + \theta_i^k \right)}_{branch} \underbrace{\sigma(w_k \cdot y + \zeta_k)}_{trunk} \right| < \epsilon$$

holds for all $u \in V$ and $y \in K_2$.

Deep Operator Network (DeepONet)

- Generalized Universal Approximation Theorem for Operator [Chen '95, Lu et al. '19]
- **Branch net**: Input $\{f(x_i)\}_{i=1}^{m}$, output: $\left[b_1, b_2, .., b_p\right]^T \in \mathbb{R}^p$
- **Trunk net**: Input y, output: $\left[t_1, t_2, .., t_p\right]^T \in \mathbb{R}^p$
- Input f is evaluated at locations $\{y_i\}_{i=1}^{m}$

$$\hat{u} = G_\theta(f)(y) = \underbrace{\sum_{i=1}^{p} b_i(f(x_1), f(x_2), \dots, f(x_m))}_{\text{branch net}} \cdot \underbrace{tr_i(y)}_{\text{trunk net}}$$



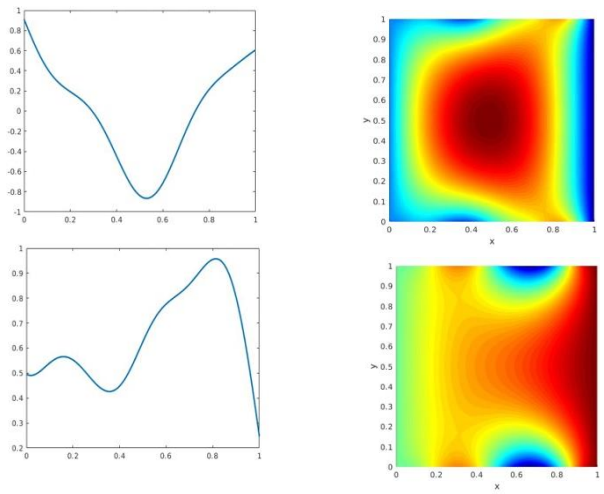
Motivation of Neural Operators

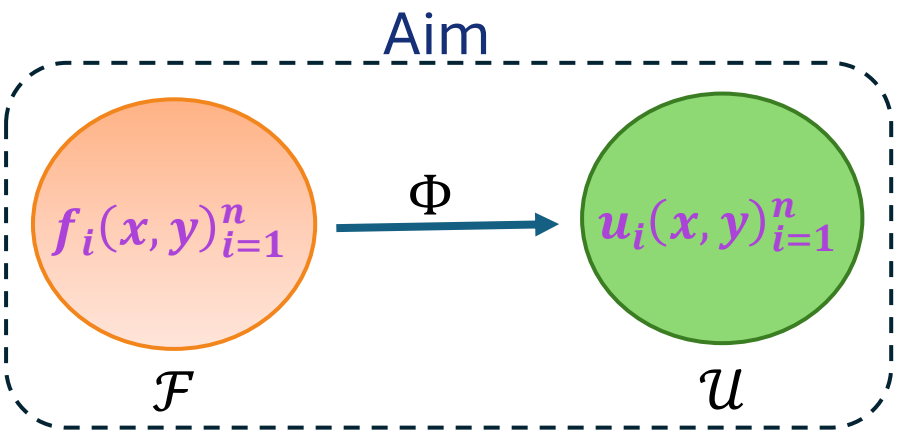
$$-\left(\frac{\partial^2 u(x,y)}{\partial x^2} + \frac{\partial^2 u(x,y)}{\partial y^2}\right) = 1 \qquad u_{\partial\Omega}(1,y) = f(x,y)$$

Domain

$$\Omega = [0,1] \times [0,1]$$

$f(x,y) \rightarrow u(x,y)$

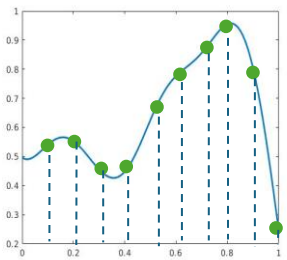


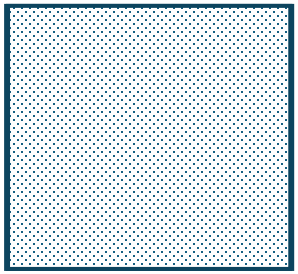


Building a DeepONet Model (Example 1)

Testing Boundary conditions

u on $\partial\Omega$

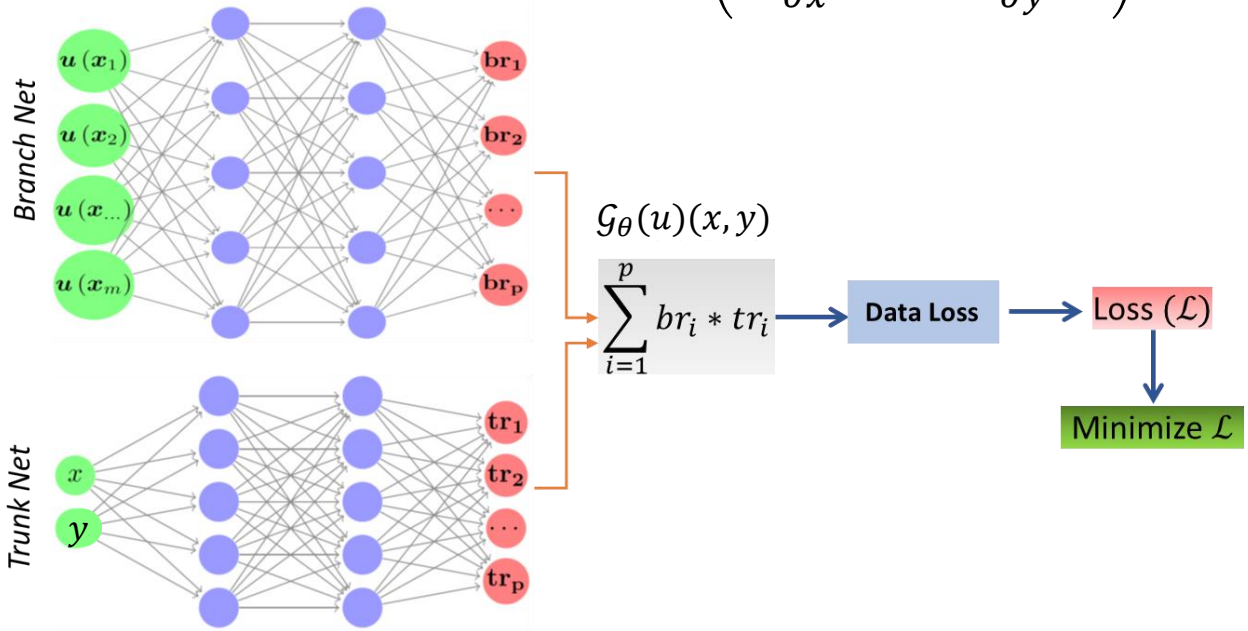




Query Points during testing

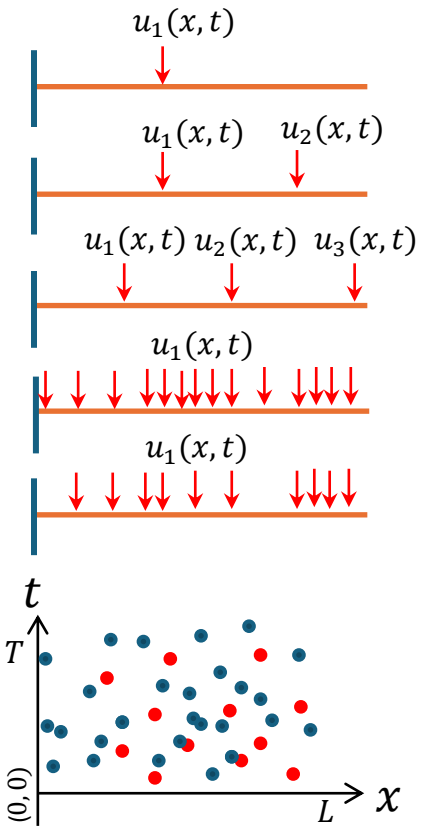
$$\mathcal{G}_\theta : u_{\partial\Omega}(0, y) \rightarrow u(x, y)$$

$$-\left(\frac{\partial^2 u(x,y)}{\partial x^2} + \frac{\partial^2 u(x,y)}{\partial y^2}\right) = 1$$

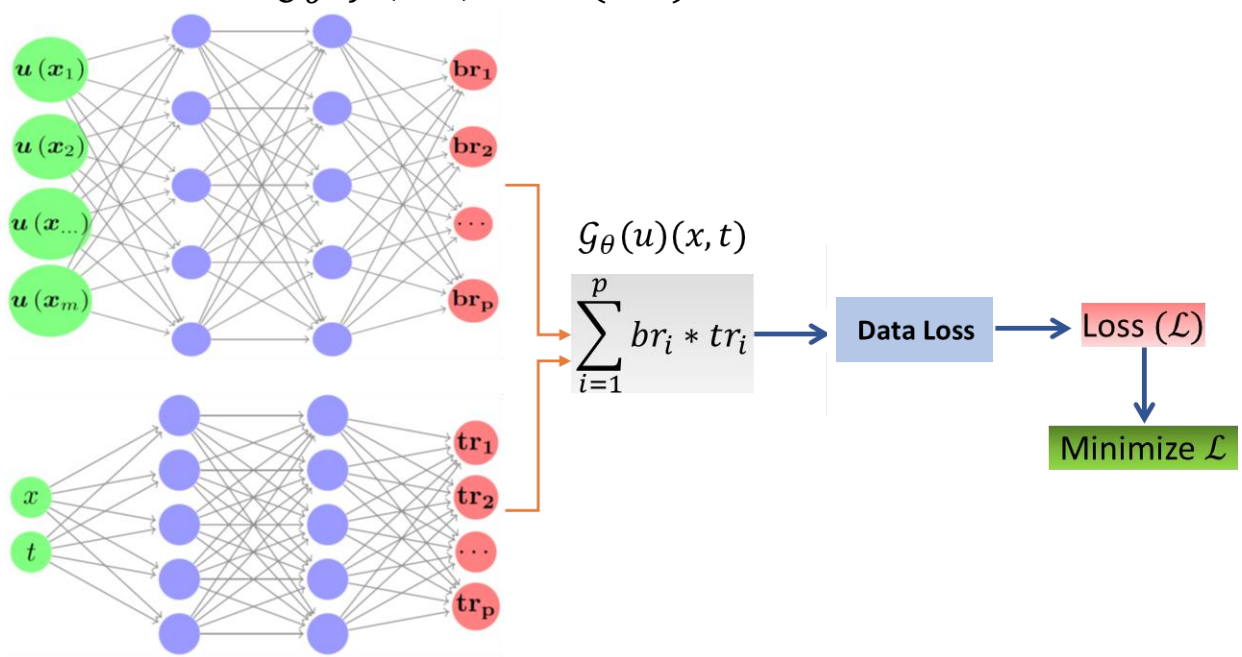


- Sensors $(m) = 10$

Building a DeepONet Model (Example 2)

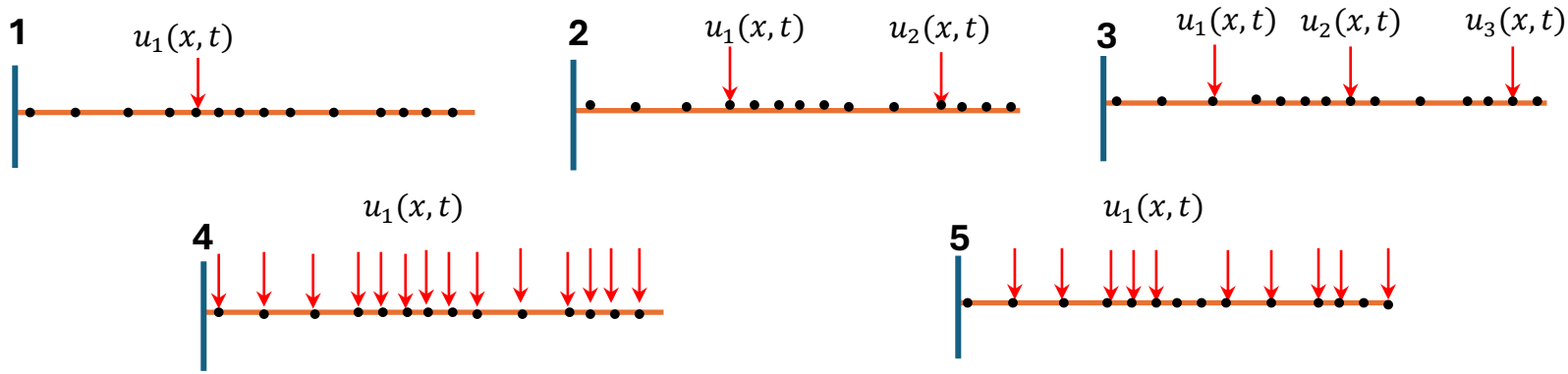


$$\mathcal{G}_\theta : f(x,t) \rightarrow w(x,t)$$



- Sensors(m) $= 15$
- Query Points

Building a DeepONet Model (Example 2)



	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12	S13	S14	S15
1	0	0	0	0	u_1	0	0	0	0	0	0	0	0	0	0
2	0	0	0	u_1	0	0	0	0	0	0	0	u_2	0	0	0
3	0	0	u_1	0	0	0	0	u_2	0	0	0	0	0	u_3	0
4	u_1	u_1	u_1	u_1	u_1	u_1	u_1	u_1	u_1	u_1	u_1	u_1	u_1	u_1	u_1
5	0	u_1	u_1	u_1	u_1	u_1	0	0	u_1	u_1	u_1	u_1	u_1	0	u_1

Fourier neural operator (FNO)

Iterative algorithm:

$$v_{t+1}(x) = \sigma\big(W v_t(x) + (\mathcal{K}(\alpha;\varphi)v_t)(x)\big)$$

for $t = 1, \dots, T-1$

Fourier integral operator:

$$(\mathcal{K}(\alpha;\varphi)v_t)(x) = \mathcal{F}^{-1}\left(R_\varphi \cdot (\mathcal{F}v_t)\right)(x)$$

$\forall x \in \Omega$

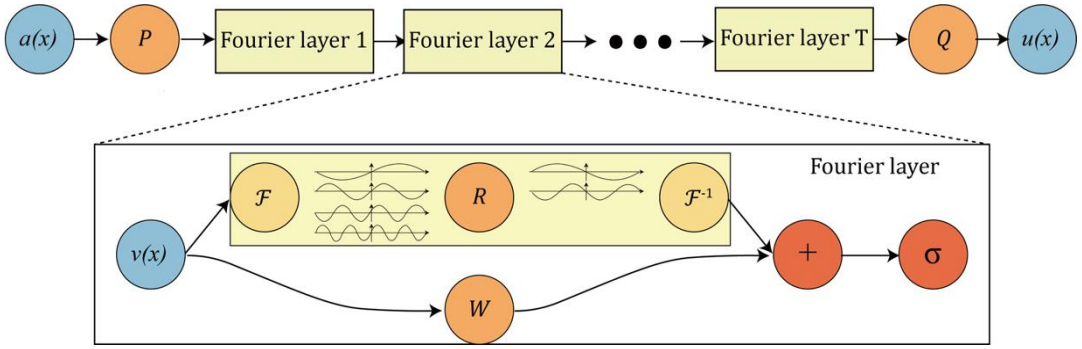
Fourier transform of function $f: \Omega \to \mathbb{R}^{d_v}$

$$(\mathcal{F}f)_j(k) = \int_\Omega f_j(x) e^{-2i\pi\langle x,k\rangle} dx$$

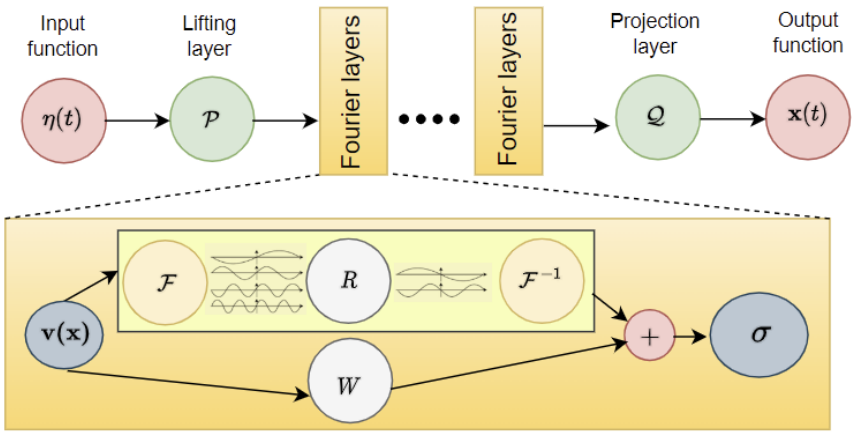
$$(\mathcal{F}^{-1}f)_j(x) = \int_\Omega f_j(k) e^{2i\pi\langle x,k\rangle} dk$$

for $j = 1, \dots, d_v$ $\quad k$: FFT modes

- Employs convolution via fast Fourier transform (FFT)
- The Fourier layer just consists of three steps:
  - ✓ Fourier transform $\mathcal{F}$
  - ✓ Linear transform on the lower Fourier modes, $\mathcal{R}$
  - ✓ Inverse Fourier transform $\mathcal{F}^{-1}$
- For time-dependent PDEs (2D in space, 1D in time):
  1. **FNO-3D**: 3D → 3D (3D convolutions)
  2. **FNO-2D**: 2D → 2D (2D + RNN)



Fourier Neural Operator



Advantage

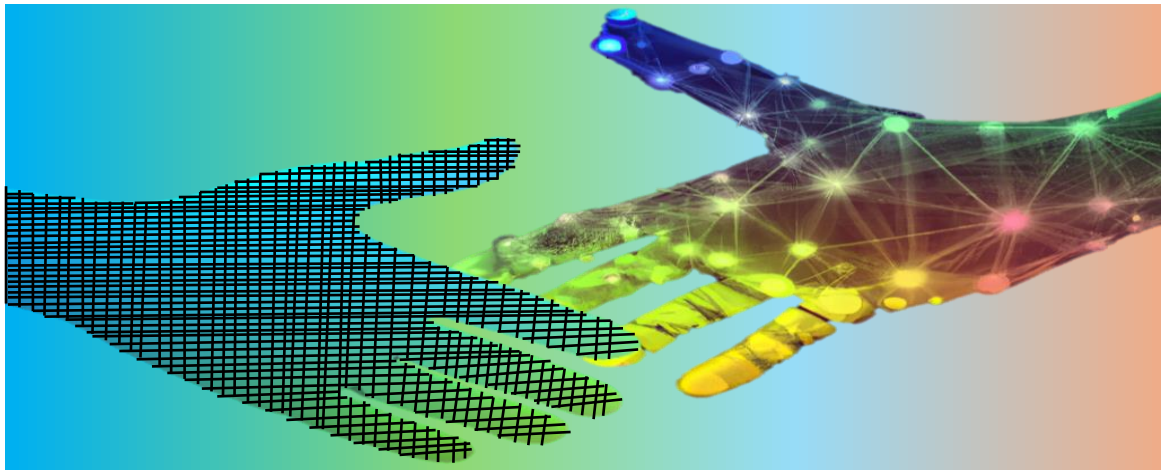
- Cheap computational cost due to FFT.
- A small size of training samples.

Limitation:

- For FFT, the input is assumed to be periodic.
- Only capture steady-state responses.

Closing Remarks

SciML: Best of both worlds



Numerical Solvers

Slow
Error guarantees
Few parameters (e.g. RK)
Large inductive bias
No data
Generalization?

Machine Learning Solvers

Fast
No error guarantees
Lots of parameters (e.g. deep NN)
Little inductive bias
Lots of data
Generalization?